Cheat Sheet

Input & Output

1. print ("Hello", "World")
   output : Hello World
2. print ("=" * 30)
   output : print "=" 30 times
3. name = value
   "=" sign assigns a value to a variable.
4. type (x)
   Returns the type of a value
   str (text) int (whole number) float (decimal number)
5. Input ("...")
   It always return text string
6. int (x) : Convert to an integer
   float (x) : Converts to a decimal
7. f"{x:.2f}" → displays number with 2 decimal places
   f"{x:>10}" → right aligns the value in a field with 10 character wide.

137 // 60 → 2 (Quotient)
137 % 60 → 17 (Remainder)

(b) > Input () always give you the input data from the user and it is important to collect data
> "20" is the data stored in a string compared to 20 which is just a number
% is the modulo operator which is used to find the remainder after dividing one number by another. It can be used for algorithm and mathematics tasks.

01

1. **Comparisons**
   == [Equals to, which gives a true statement]
   != [Not equal to, gives a true statement]
   < [is less than, gives a false statement]
   > [is greater than, gives a true statement]
   <= [is less or equal to, a false statement]
   >= [is greater or equal to, a true statement]

== → Is the equality operator / ask question
= → is assigement operator to store a value in a variable / store data

2. **Combining Conditions**
   and [logical AND, both statement has to be true to get true statement]
   or [logical OR, at least one statement has to be true to get true statement]
   not [logical NOT, statement is false]

3. **Decisions**
   if [if statement, Booleans decide whether the statement is true]
   elif [checked top to bottom. The first True statement runs and the rest skipped]
   else [It runs when the 'if condition' was False]

4. **Indentation**
   Intended lines are the block which needs to be respected in python. In loops for examples while using if statement.

5. **While loops**
   It repeats the block while condition is True
   It stops when condition becomes false (count += 1)
   while True : + break → it loops until something inside the loop decides to stop instead of counting in advance.
   break and continue → the break exits the loop and continue skips the rest of the pass and goes straight to checking the condition

6. **For loops**
   range (stop) → when we know how many times to repeat
   range (3) → starts from 0 to 3
   range (start, stop, step) → Counting by more than one at a time, counting with a negative step
   Enumerate ( ) → It hands the position and the value together

7. **For or while**
   know the count in advance → For
   waiting on a condition or a signal → while

02

1. **def name (parameters)**
   defining creates a function, calling runs it. (ex: def greet ( ): print ("record check") greet ( ))
   defining creates a function, calling runs it. greet ( )

2. **return vs print return**
   return : gives something back to use
   print ( ) : Shows the output

3. **Argument : by position, by keyword & default value**
   position → values are passed in the same order as parameters are defined in the function (1st value → 1st parameter, 2nd value → 2nd parameter)
   keyword → pass values by parameter names while calling the function (order does not matter, more readable, less error prone)

4. ***args and **kwargs : any number of values or of named value**
   *args : collects any extra positional arguments into a tuple
   **kwargs : collects extra keyword arguments into a dict

5. **Scope**
   - A function's own variable do not leak out.
   - A variable created inside a function only exists inside it (= local : local parameters
   - return is not limited to 1 value, Separate them with commas
     Ex return difference, percent
   Function : Everything needs to come in a parameter, and what is provided goes out via return

6. **Docstrings**
   - Used to document python, provide a description of what a module, class, function or method does
   - declared in triple quotes (""" or """)

7. **Modules**
   - Import brings in a whole new module Ex (print (math.pi))
   - Accesses what is inside it with a dot

8. **if __name__ == "__main__"**
   - only run this part when this file is launched directly

03

List
Stores multiple items in a single variable
list = ["apple", "cherry", "peach", "Banana"]

Indexing
returns the position at the first occurance of the specified value
list.index (element, start, end)
x = fruit.index ("peach", 1)

Slicing
- Specify the start index and end index seperated by a colon, to return part of a string
  x = "Cherry"
  print (x[1:4])

.append ( )
- add an element to end of list
  x = ["1", "2", "3"]
  x.append ("4")

.remove ( )
- removes an element from list
  x.remove ("2")

List comprehension
- offers shorter syntax when a new file has to be created
  list = ["apple", "cherry", "peach", "Banana"]
  newlist = [x for x in list if "a" in x]

Tuple
- collection which is ordered and unchangeable
  tuple1 = ("x", "y", "z"
  print (tuple1)

Dictionaries
- Used to store data values in key : value pairs
  dict = {
  "brand" : "Bmw"
  "Origin" : "Germany"
  }
  print (dict) → items
  print (dict["brand"]) → "Bmw" / x = dict.get ("origin")

Looping over a dict
- values ( ) → method to return values of a dictionary
  for x in dict.keys ( ) : print (x)
- items ( ) → Loop through both keys & values
  for x, y in dict.items ( ) : print (x, y)

Set
- stores unique values only
  n = {1, 2, 4, 4}
  print (n) # 1, 2, 4

File
- allow a program to Save / read information
  with open ("record.txt", "w") as f : # Write to a file
  f.write ("R-001, 80, 100\n")
  with open ("record.txt", "r") as f : # line by line
  for line in f :
  x = line.strip ( ).split (",")

Handling errors
- try & except allow a program to handle expected errors instead of stopping unexpectedly.
  text = "abc"
  try :
  n = float (text)
  print (n)
  except ValueError :
  print ("Not valid")
  output : Not valid
  try :
  with open ("missing.txt", "r") as f :
  print (f.read ( ))
  except FileNotFoundError :
  print ("File does not exist")
  File does not exist

List of dictionaries
- stores multiple record, each one has named fields
  records = [
  {"label" : N01, "Value" : 85 "limit" : 100}
  {"label" : N02, "Value" : 95 "limit" : 100}
  {"label" : N03, "Value" : 110 "limit" : 100}
  ]
  for record in records :
  print (record ['label'], record ["value"])

04

A class defines the shape __init__ fills it in

```
class Record :
  def __init__ (self, label, value, limit) :
    self.label = label
    self.value = value
    self.limit = limit
r1 = Record ("Survey_01", 90.0, 200.0)
print (r1.label)
```

Self refers to the particular object which is used, to access the object's data

```
class Record :
  def __init__ (self, label) :
    self.label = label
```

Attributes are variables stored inside an object

```
class Record :
  def __init__ (self, value) :
    self.value = value
r1 = Record (60)
print (r1.value)
```

Objects are Separate
- Each object has its own attributes

```
r1 = Record (60)
r2 = Record (90)
r1.value = 60
```

Methods
- It is a function written inside a class

```
class Record :
  def Show (self) :
    print ("MyName")
r1 = Record (60)
r1.Show ( )
```

Methods can change their own object

```
Class Counter :
  def __init__ (self) :
    Self.count = 0
  def increase (self) :
    Self.count += 1
C1 = Counter ( )
C1.increase ( )
```

Class Level Attributes
- It is shared by all objects of the class.
- written directly inside the class

```
Class Record :
  category = "workshop"
r1 = Record (60)
r2 = Record (80)
```

A list of object
- Store several objects in a list, loop through them.

```
records = [ ]
with open ("records.txt", "r") as File :
  for line in File :
    label, value = line.Strip ( ).split (",")
    records.append (Record (Float (value)))
```